



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #10
Portable Program Development***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Portable Program Development**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Portable Program Development(PPD) refers to **writing solution or software that can run across multiple platforms or operating systems with minimal or no changes**. It focuses on creating **platform-independent code by following standard conventions, using portable libraries, and avoiding system-specific features**. A key part of portable programming is **writing solutions for well-defined problem statements in a way that they remain functional and consistent across environments**. This requires clear problem analysis, modular design, and careful consideration of dependencies. By developing solutions with portability in mind, developers ensure that their programs are **adaptable, maintainable, and scalable across diverse systems such as Windows, Linux, macOS, and embedded platforms**.

Benefits of PPD

1. **Wider Compatibility:** Programs can run on multiple platforms (Windows, Linux, macOS, etc.) without modification, increasing their reach.
2. **Cost-Effective:** Reduces the need for separate development and maintenance teams for different platforms.
3. **Time-Saving:** Saves time in development and testing by using a single codebase for multiple systems.
4. **Ease of Maintenance:** Bugs fixed or features added in one place benefit all platforms, making updates more efficient.
5. **Scalability:** Easier to scale or migrate software to new hardware or operating systems.
6. **Enhanced Code Reusability:** Encourages modular and clean coding practices, making components reusable across projects.
7. **Future-Proofing:** Programs remain viable even if the target environment changes, as long as the standard tools/libraries remain supported.
8. **Better Collaboration:** Teams working on different platforms can collaborate more easily when the code is portable.

To achieve true portability, programs often need to adapt their behavior based on the underlying system. This is where **platform-specific macros** become useful.

Using Platform-Specific Macros

Most compilers define special macros that allow the code to **detect the platform at preprocessing time** and include or exclude system-specific sections accordingly. To support cross-platform development, these macros can be used with conditional compilation directives (like `#ifdef`) to include or exclude platform-specific code. The table below lists some of the most commonly used platform-detection macros:

Macro	Purpose / Description	Typical Platform
<code>__MINGW32__</code>	Defined when using MinGW compiler	Windows (MinGW)
<code>__WIN32</code>	Defined on 32-bit and 64-bit Windows	Windows
<code>__WIN64</code>	Defined on 64-bit Windows only	Windows
<code>__unix__</code>	Defined for Unix-like systems	Unix, Linux
<code>__linux__</code>	Specifically defined for Linux systems	Linux
<code>__APPLE__</code>	Defined when compiling on Apple/macOS	macOS
<code>__ANDROID__</code>	Defined when compiling for Android	Android
<code>__CYGWIN__</code>	Defined for Cygwin environment on Windows	Windows (Cygwin)

We typically use the following format to write platform-specific code in a portable program. This ensures that the **correct code block is compiled based on the target operating system**. This approach enhances cross-platform compatibility by allowing conditional compilation based on predefined system macros.

```
#ifdef __MINGW32__
    // Windows-specific code
#elif defined(__unix__)
    // Unix/Linux-specific code
#elif defined(__APPLE__)
    // macOS-specific code
#endif
```

To inspect the macros defined by the compiler for a given system or environment, we can use the **gcc -E** option, which runs only the preprocessing phase of compilation and displays the output of preprocessing on the standard output device.

Coding Example_1: Portable Code to Print OS Name

```
#include <stdio.h>
int main() {
    printf("Running on: ");

#ifdef __MINGW32__
    printf("Windows (MinGW)\n");
#endif

#ifdef _WIN32
    printf("Windows\n");
#endif

#ifdef _WIN64
    printf("Windows 64-bit\n");
#endif

#ifdef __unix__
    printf("Unix-like system\n");
#endif

#ifdef __linux__
    printf("Linux\n");
#endif

#ifdef __APPLE__
    printf("macOS\n");
#endif

#ifdef __ANDROID__
    printf("Android\n");
#endif

    return 0;
}
```

Running this code on Mingw Windows 32 system.

Preprocessing stage output is:

```
int main() {
    printf("Running on: ");

    printf("Windows (MinGW)\n");

    printf("Windows\n");
# 36 "check.c"
    return 0;
}
```

Execution output:

```
C:\Users\Dell>a
Running on: Windows (MinGW)
Windows
```

Coding Example_2 : Clearing the screen on different platforms

```
#include <stdio.h>
#include <stdlib.h>

void clearScreen() {
#ifdef _WIN32
    system("cls"); // Windows
#endif

#ifdef __unix__
    system("clear"); // Unix/Linux
#endif

#ifdef __APPLE__
    system("clear"); // macOS
#endif
}

int main() {
    printf("This message will be cleared in 3 seconds...\n");

#ifdef _WIN32
    system("timeout /T 3 > nul");
//Command to wait for three seconds
//in windows
#endif

#ifdef __unix__
    system("sleep 3");
#endif

#ifdef __APPLE__
    system("sleep 3");
#endif

    clearScreen();

    printf("Screen cleared!\n");
    return 0;
}
```

**Running this code on Mingw32 Windows system.
Preprocessing output:**

```
void clearScreen() {
    system("cls");
# 16 "check.c"
}

int main() {
    printf("This message will be cleared in 3 seconds...\n");

    system("timeout /T 3 > nul");
# 33 "check.c"
    clearScreen();

    printf("Screen cleared!\n");
    return 0;
}
```

**Execution Output: Once after you run the code,
screen gets cleared in 3 seconds**

```
Screen cleared!
C:\Users\De11>
```

Coding Example_3 : Portable code for Character user input and display.**Version 1: Writing a separate block of code for each OS**

```
#include<stdio.h>
int main()
{
    #ifdef __MINGW32__
        printf("Windows\n Enter the character\n");
        char ch1 = getchar();
        printf("Enter another character\n");
        fflush(stdin);
        char ch2 = getchar();
        printf("You typed %c and %c\n", ch1, ch2);
    #endif

    #ifdef __unix__
        printf("UNIX\nEnter the character\n");
        char ch1 = getchar();
        printf("Enter another character\n");
        #include<stdio_ext.h>
        __fpurge(stdin);
        char ch2 = getchar();
        printf("You typed %c and %c\n", ch1, ch2);
    #endif

    #ifdef __APPLE__
        printf("MAC\nEnter the character\n");
        char ch1 = getchar();
        printf("Enter another character\n");
        fpurge(stdin);
        char ch2 = getchar();
        printf("You typed %c and %c\n", ch1, ch2);
    #endif
    return 0 ;
}
```

**Run on Mingw32 Windows system.
Preprocessing stage output:**

```
# 2 "check.c"
int main()
{
    printf("Windows\nEnter the character\n");
    char ch1 = getchar();
    printf("Enter another character\n");
    fflush(
        (&iob[0])
    );
    # 8 "check.c" 3
    # 8 "check.c"
    char ch2 = getchar();
    printf("You typed %c and %c\n", ch1, ch2);
    # 34 "check.c"
}
```

Execution output:

```
C:\Users\Dell>a
Windows
Enter the character
s
Enter another character
i
You typed s and i
```

Version 2: Using a parameterized macro

```
#ifdef __WIN32
    #define CLEAR_INPUT_BUFFER() fflush(stdin)
#elif __APPLE__
    #include <stdio.h>
    #define CLEAR_INPUT_BUFFER() fpurge(stdin)
#elif __unix__
    #include <stdio.h>
    #define CLEAR_INPUT_BUFFER() __fpurge(stdin)
#endif
```

```
int main() {
    char ch1, ch2;

    // First character input
    printf("Enter the first character: ");
    scanf("%c", &ch1);
    CLEAR_INPUT_BUFFER();

    // Second character input
    printf("Enter the second character: ");
    scanf("%c", &ch2);
    CLEAR_INPUT_BUFFER();

    // Display characters
    printf("You entered: '%c' and '%c'\n", ch1, ch2);

    return 0;
}
```

Preprocessing output:

```
# 16 "check.c"
int main() {
    char ch1, ch2;

    printf("Enter the first character: ");
    scanf("%c", &ch1);
    fflush(
# 22 "check.c" 3
    (&_iob[0])
# 22 "check.c"
    );

    printf("Enter the second character: ");
    scanf("%c", &ch2);
    fflush(
# 27 "check.c" 3
    (&_iob[0])
# 27 "check.c"
    );

    printf("You entered: '%c' and '%c'\n", ch1, ch2);

    return 0;
}
```

Execution output:

```
C:\Users\Dell>a
Enter the first character: s
Enter the second character: i
You entered: 's' and 'i'
```

Try it-> Explore #elif defined() to rewrite the above code more compactly !

Portability = Possibility in C!!